

IMU to Quaternion to RPY

ESP32 + QMI8658 + Madgwick and Mahony filters

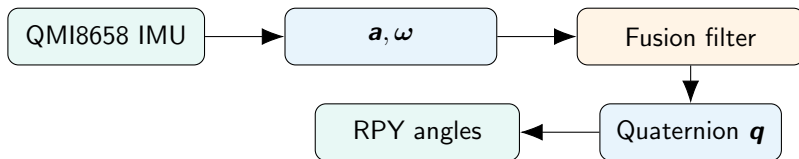
Medical Robotics / Spatial Mathematics Demo

$$\boxed{(a_x, a_y, a_z, g_x, g_y, g_z)} \implies \boxed{q_w, q_x, q_y, q_z} \implies \boxed{\phi, \theta, \psi}$$

Learning goal

By the end of this demo, we will learn:

- 1 What the accelerometer measures: a vector related to gravity and motion.
- 2 What the gyroscope measures: angular velocity.
- 3 Why orientation is estimated internally as a quaternion.
- 4 How Madgwick combines gyro prediction with accelerometer correction.
- 5 How Mahony interprets the same problem as feedback control.
- 6 How a quaternion is converted into roll, pitch, and yaw.
- 7 Why yaw is only relative with a 6-axis IMU.



Roll, pitch, and yaw convention

Axis naming convention

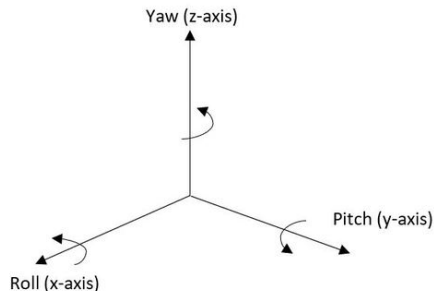
ϕ = roll about X ,

θ = pitch about Y ,

ψ = yaw about Z .

Sign convention

The positive rotation direction is defined by the right-hand rule. Arrow direction must be interpreted together with the chosen body/world frame convention.



In this course, RPY describes the **body-frame orientation relative to the world frame**:

$$\boxed{\mathbf{v}_W = \mathbf{R}_{WB} \mathbf{v}_B}, \quad \mathbf{R}_{WB} = \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi).$$

What is RPY relative to?

Short answer

Roll, pitch, and yaw are not angles that belong only to the global frame or only to the local frame. They describe the **orientation relationship between two frames**.

Local/body frame B

- Attached to the ESP32-S3-Matrix board.
- Moves and rotates with the QMI8658 sensor.
- Raw IMU readings are measured in this frame:

$$\mathbf{a}_B, \quad \boldsymbol{\omega}_B.$$

Global/world frame W

- Reference frame used by the Python viewer.
- The vertical direction is established by gravity.
- The quaternion estimates how B is oriented relative to W .

$$\mathbf{R}_{WB} : \text{body } B \longrightarrow \text{world } W$$

Body-to-world orientation and the RPY convention

Goal: estimate the orientation of the moving body frame B relative to the fixed world frame W .

Body-to-world rotation matrix

We use the ZYX Euler-angle convention:

$$\mathbf{R}_{WB} = \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi)$$

with

$$\phi = \text{roll}, \quad \theta = \text{pitch}, \quad \psi = \text{yaw}.$$

A vector expressed in the body frame is mapped into world coordinates by

$$\mathbf{v}_W = \mathbf{R}_{WB} \mathbf{v}_B$$

Intrinsic interpretation

The rotations are about the **moving body axes**:

yaw $\rightarrow$ pitch $\rightarrow$ roll

- 1 yaw ψ about body z ,
- 2 pitch θ about the new y axis,
- 3 roll ϕ about the new x axis.

This is called an **intrinsic ZYX rotation**.

Equivalent extrinsic interpretation

The same final orientation can also be described using **fixed world axes**:

roll $\rightarrow$ pitch $\rightarrow$ yaw

- 1 roll ϕ about fixed world X ,
- 2 pitch θ about fixed world Y ,
- 3 yaw ψ about fixed world Z .

Important interpretation

RPY is not “only global” or “only local.”

It describes the **orientation of the body frame relative to the world frame**, under a specific rotation convention.

$$\mathbf{R}_{WB} : B \longrightarrow W$$

Because matrix multiplication acts from right to left, $\mathbf{R}_x(\phi)$ acts first in the extrinsic interpretation.

What is the world frame for a 6-axis IMU?

Gravity fixes vertical, not compass heading

With only an accelerometer and gyroscope, the filter can use gravity to define the world vertical axis:

gravity $\Rightarrow Z_W$ direction.

But rotating around gravity does not change the gravity vector:

no magnetometer $\Rightarrow$ no absolute yaw reference.

Observable from gravity

ϕ = roll, θ = pitch

These are corrected long-term because tilting changes the measured gravity direction.

Relative only

ψ = yaw

Yaw is measured relative to the initial or reset heading, and it slowly drifts because there is no compass-like reference.

The six numbers from the IMU

Accelerometer

$$\mathbf{a} = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}$$

- Unit used in our firmware: g .
- When stationary, it mainly measures gravity direction.
- Good long-term reference for roll and pitch.
- Sensitive to vibration and linear acceleration.

Gyroscope

$$\boldsymbol{\omega} = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

- Unit used in our firmware: $^\circ/s$.
- Measures angular velocity around sensor axes.
- Smooth and responsive for fast motion.
- Integration accumulates drift.

Notation: in code, g_x, g_y, g_z may mean gyro readings; in the math we write them as $\omega_x, \omega_y, \omega_z$ to avoid confusing gyroscope with gravity g .

From raw registers to physical quantities

The IMU gives signed 16-bit integers. The first step is scaling them.

Raw-to-physical conversion

For accelerometer range $\pm 4g$:

$$a_i = \frac{4}{32768} a_{i,\text{raw}}, \quad i \in \{x, y, z\}.$$

For gyroscope range $\pm 1024^\circ/\text{s}$:

$$\omega_i = \frac{1024}{32768} g_{i,\text{raw}}, \quad i \in \{x, y, z\}.$$

Firmware idea

```
ax = raw_ax * (4.0 / 32768.0);  
ay = raw_ay * (4.0 / 32768.0);  
az = raw_az * (4.0 / 32768.0);  
  
gx = raw_gx * (1024.0 / 32768.0);  
gy = raw_gy * (1024.0 / 32768.0);  
gz = raw_gz * (1024.0 / 32768.0);
```

Two coordinate frames: body frame and world frame

Body frame B

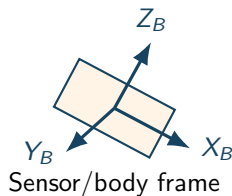
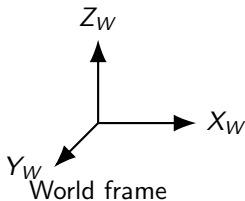
- Attached to the sensor board.
- Axes move when the board moves.
- IMU measurements are naturally expressed here:

$$\mathbf{a}_B, \quad \boldsymbol{\omega}_B.$$

World frame W

- Fixed reference frame used by the viewer.
- Gravity points along the vertical axis.
- Orientation tells us how to rotate B into W :

$$\mathbf{v}_W = \mathbf{R}_{WB} \mathbf{v}_B.$$



Accelerometer: gravity as a direction reference

When the board is approximately stationary, the accelerometer vector mostly points along gravity.

Normalize the measured acceleration

$$\hat{\mathbf{a}} = \frac{\mathbf{a}}{\|\mathbf{a}\|} = \frac{1}{\sqrt{a_x^2 + a_y^2 + a_z^2}} \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}.$$

Direct roll/pitch intuition

$$\phi_{acc} = \text{atan2}(a_y, a_z)$$

$$\theta_{acc} = \text{atan2}\left(-a_x, \sqrt{a_y^2 + a_z^2}\right)$$

What gravity cannot tell us

Gravity gives an “up/down” direction. It does not tell us compass heading. Therefore, yaw is not observable from accelerometer alone.

Gyroscope: angular velocity and drift

The gyroscope gives angular velocity around the body axes:

$$\boldsymbol{\omega}_B = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}.$$

Simple integration idea

For one axis, the intuitive formula is

$$\alpha_{k+1} = \alpha_k + \omega_k \Delta t.$$

This is fast and smooth.

But bias causes drift

If the measured rate has a small bias,

$$\omega_{meas} = \omega_{true} + b,$$

then angle error grows approximately as

$$\alpha_{error}(t) \approx bt.$$

Gyro is excellent for short-term motion; poor as a long-term reference.

Why not store orientation directly as roll, pitch, yaw?

Euler angles are human-readable

$$(\phi, \theta, \psi) = (\text{roll}, \text{pitch}, \text{yaw})$$

- Easy to display.
- Easy to discuss in class.
- Matches our serial output.

But awkward for integration

- Rotations do not commute.
- Singularities can occur.
- Updating Euler angles directly becomes messy in 3D.

Therefore

The filter keeps orientation internally as a quaternion, then converts it to roll-pitch-yaw only for display.

Quaternion representation

A quaternion stores 3D orientation with four numbers:

$$\mathbf{q} = [q_w \quad q_x \quad q_y \quad q_z]^T = [q_1 \quad q_2 \quad q_3 \quad q_4]^T.$$

Unit quaternion constraint

$$\|\mathbf{q}\| = 1, \quad q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1.$$

Conceptual meaning

A quaternion represents a 3D rotation without needing three separate angle updates.

Practical filter habit

After every update, normalize:

$$\mathbf{q} \leftarrow \frac{\mathbf{q}}{\|\mathbf{q}\|}.$$

This prevents numerical growth or shrinkage.

Gyro prediction in quaternion form

Convert gyroscope from degrees/s to radians/s: $\omega_{rad/s} = \omega_{deg/s} \frac{\pi}{180}$.

Quaternion derivative from angular velocity

$$\dot{\mathbf{q}}_\omega = \frac{1}{2} \mathbf{q} \otimes [0 \quad \omega_x \quad \omega_y \quad \omega_z]^T.$$

Expanded for $\mathbf{q} = [q_w, q_x, q_y, q_z]^T$:

$$\begin{aligned}\dot{q}_w &= \frac{1}{2}(-q_x\omega_x - q_y\omega_y - q_z\omega_z), \\ \dot{q}_x &= \frac{1}{2}(q_w\omega_x + q_y\omega_z - q_z\omega_y), \\ \dot{q}_y &= \frac{1}{2}(q_w\omega_y - q_x\omega_z + q_z\omega_x), \\ \dot{q}_z &= \frac{1}{2}(q_w\omega_z + q_x\omega_y - q_y\omega_x).\end{aligned}$$

Discrete update: prediction step

With sampling period Δt , the simplest update is Euler integration:

$$\mathbf{q}_{pred,k+1} = \mathbf{q}_k + \dot{\mathbf{q}}_{\omega,k} \Delta t.$$

Then normalize:

$$\mathbf{q}_{pred,k+1} \leftarrow \frac{\mathbf{q}_{pred,k+1}}{\|\mathbf{q}_{pred,k+1}\|}.$$

Problem

This is still only gyro integration. It is smooth, but it drifts.

What we need next

Use the accelerometer to correct the predicted quaternion so that the estimated gravity direction matches the measured gravity direction.

Two correction philosophies

After gyro prediction, both filters use the accelerometer as a gravity reference.

Madgwick

$$\dot{\mathbf{q}} = \dot{\mathbf{q}}_{\omega} - \beta \mathbf{s}$$

- Treats correction as an optimization problem.
- Uses a normalized gradient direction $\mathbf{s}$.
- Gain β controls correction strength.

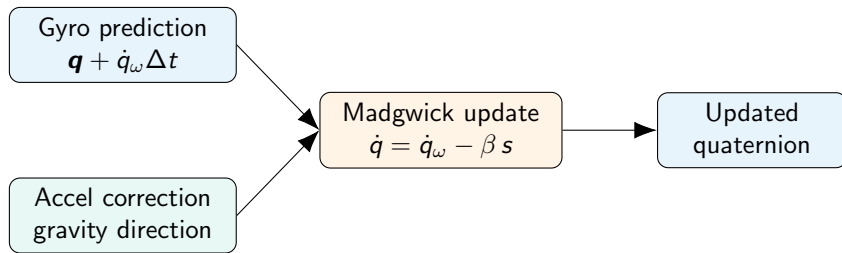
Mahony

$$\boldsymbol{\omega}_{corr} = \boldsymbol{\omega} + K_P \mathbf{e} + K_I \int \mathbf{e} dt$$

- Treats correction as feedback control.
- Uses gravity-direction error $\mathbf{e}$.
- Gains K_P, K_I act like PI control.

Same sensor inputs, same quaternion output, different correction style.

The Madgwick idea



One-line conceptual equation

$$\dot{\mathbf{q}} = \underbrace{\dot{\mathbf{q}}_{\omega}}_{\text{gyro prediction}} - \underbrace{\beta \mathbf{s}}_{\text{gravity correction}}$$

- $\mathbf{s}$ is the normalized gradient direction that reduces gravity mismatch.
- β controls how strongly accelerometer correction pulls the estimate.

Madgwick equations 1/3: IMU data and gyro prediction

Inputs at sample k

$$\mathbf{a}_{B,k} = [a_x \quad a_y \quad a_z]^T, \quad \boldsymbol{\omega}_{B,k} = [\omega_x \quad \omega_y \quad \omega_z]^T, \quad \mathbf{q}_k = [q_w \quad q_x \quad q_y \quad q_z]^T.$$

1. Normalize acceleration

When the board is approximately stationary, acceleration mainly gives gravity direction:

$$\hat{\mathbf{a}}_{B,k} = \frac{\mathbf{a}_{B,k}}{\|\mathbf{a}_{B,k}\|} = \frac{1}{\sqrt{a_x^2 + a_y^2 + a_z^2}} \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}.$$

2. Gyro predicts quaternion motion

Convert $\omega_x, \omega_y, \omega_z$ to rad/s, then

$$\dot{\mathbf{q}}_{\omega,k} = \frac{1}{2} \mathbf{q}_k \otimes \begin{bmatrix} 0 \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}.$$

Expanded:

$$\dot{\mathbf{q}}_{\omega,k} = \frac{1}{2} \begin{bmatrix} -q_x\omega_x - q_y\omega_y - q_z\omega_z \\ q_w\omega_x + q_y\omega_z - q_z\omega_y \\ q_w\omega_y - q_x\omega_z + q_z\omega_x \\ q_w\omega_z + q_x\omega_y - q_y\omega_x \end{bmatrix}.$$

Meaning: gyro prediction is fast and smooth, but still drifts over time.

Madgwick equations 2/3: gravity correction direction

Madgwick corrects the gyro prediction by comparing measured gravity with predicted gravity.

Predicted gravity from quaternion

For $\mathbf{q} = [q_w, q_x, q_y, q_z]^T$:

$$\hat{\mathbf{g}}_B(\mathbf{q}_k) = \begin{bmatrix} 2(q_x q_z - q_w q_y) \\ 2(q_w q_x + q_y q_z) \\ 1 - 2(q_x^2 + q_y^2) \end{bmatrix}.$$

Gravity mismatch:

$$\mathbf{f}(\mathbf{q}_k, \hat{\mathbf{a}}_B) = \begin{bmatrix} 2(q_x q_z - q_w q_y) - \hat{a}_x \\ 2(q_w q_x + q_y q_z) - \hat{a}_y \\ 1 - 2(q_x^2 + q_y^2) - \hat{a}_z \end{bmatrix}.$$

Gradient direction

Jacobian of the gravity mismatch:

$$\mathbf{J} = \begin{bmatrix} -2q_y & 2q_z & -2q_w & 2q_x \\ 2q_x & 2q_w & 2q_z & 2q_y \\ 0 & -4q_x & -4q_y & 0 \end{bmatrix}.$$

Correction direction:

$$\mathbf{s}_k = \mathbf{J}^T \mathbf{f}, \quad \hat{\mathbf{s}}_k = \frac{\mathbf{s}_k}{\|\mathbf{s}_k\|}.$$

Classroom interpretation

$\hat{\mathbf{s}}_k$ points in the quaternion-space direction that makes predicted gravity closer to measured gravity.

Madgwick equations 3/3: obtain the four quaternion values

Now combine the fast gyro prediction with the accelerometer-based correction.

Corrected quaternion derivative

$$\dot{\mathbf{q}}_k = \dot{\mathbf{q}}_{\omega,k} - \beta \hat{\mathbf{s}}_k$$

where β controls how strongly gravity correction pulls the estimate.

Use the sampling interval

If timestamps are available,

$$\Delta t = t_k - t_{k-1}.$$

Otherwise, for a fixed 100 Hz task,

$$\Delta t \approx 0.01 \text{ s}.$$

Integrate and normalize

$$\mathbf{q}_{k+1}^* = \mathbf{q}_k + \dot{\mathbf{q}}_k \Delta t$$

$$\mathbf{q}_{k+1} = \frac{\mathbf{q}_{k+1}^*}{\|\mathbf{q}_{k+1}^*\|}$$

Output of one update

$$(a_x, a_y, a_z, \omega_x, \omega_y, \omega_z, \mathbf{q}_k, \Delta t) \longrightarrow \mathbf{q}_{k+1} = [q_w \quad q_x \quad q_y \quad q_z]_{k+1}^T.$$

Choosing the gain β

Small β

- Trusts the gyroscope more.
- Smoother short-term motion.
- Corrects drift more slowly.

Large β

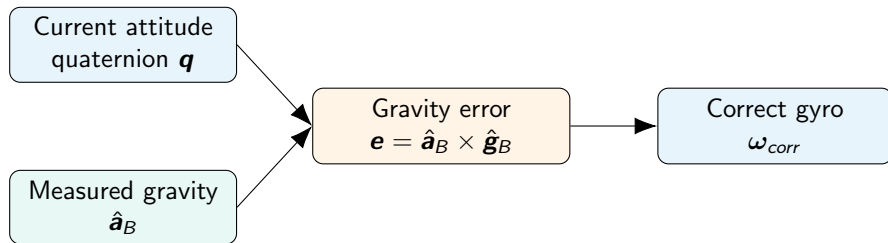
- Trusts accelerometer correction more.
- Faster convergence to gravity.
- More sensitive to vibration and linear acceleration.

Experiments

Let us change β and observe the viewer. The filter becomes a tunable physical system, not just an equation.

Our demo starts with $\beta = 0.10$.

The Mahony idea



Feedback-control view

Mahony computes an orientation error from gravity direction, then feeds that error back into the gyroscope:

$$\omega_{corr} = \omega + K_P \mathbf{e} + K_I \mathbf{b}$$

where $\mathbf{b}$ is the accumulated integral correction.

Mahony gravity error

First, normalize the measured acceleration:

$$\hat{\mathbf{a}}_B = \frac{\mathbf{a}_B}{\|\mathbf{a}_B\|}.$$

The quaternion predicts what gravity should look like in the body frame:

$$\hat{\mathbf{g}}_B(\mathbf{q}) = \begin{bmatrix} 2(q_x q_z - q_w q_y) \\ 2(q_w q_x + q_y q_z) \\ q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix}.$$

Error vector

$$\mathbf{e} = \hat{\mathbf{a}}_B \times \hat{\mathbf{g}}_B(\mathbf{q}).$$

Intuition

If predicted gravity and measured gravity point in the same direction, then $\mathbf{e} \approx \mathbf{0}$ and no correction is needed.

Mahony as PI feedback on angular velocity

Integral correction

$$\mathbf{b}_{k+1} = \mathbf{b}_k + \mathbf{e}_k \Delta t.$$

Corrected angular velocity

$$\boldsymbol{\omega}_{corr,k} = \boldsymbol{\omega}_k + K_P \mathbf{e}_k + K_I \mathbf{b}_k.$$

Then the quaternion is propagated using the corrected angular velocity:

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \begin{bmatrix} 0 & \omega_{x,corr} & \omega_{y,corr} & \omega_{z,corr} \end{bmatrix}^T.$$

Mahony looks very familiar to control: it is PI correction.

At each sample k

- 1 Read six IMU values:

$$a_x, a_y, a_z, \omega_x, \omega_y, \omega_z.$$

- 2 Normalize acceleration: $\hat{\mathbf{a}} = \mathbf{a} / \|\mathbf{a}\|$.

- 3 Use current quaternion to predict gravity $\hat{\mathbf{g}}_B(\mathbf{q})$.

- 4 Compute feedback error:

$$\mathbf{e} = \hat{\mathbf{a}}_B \times \hat{\mathbf{g}}_B(\mathbf{q}).$$

- 5 Correct gyroscope:

$$\boldsymbol{\omega}_{\text{corr}} = \boldsymbol{\omega} + K_P \mathbf{e} + K_I \int \mathbf{e} dt.$$

- 6 Integrate quaternion and normalize.

Choosing Mahony gains K_P and K_I

K_P : proportional correction

- Pulls orientation back toward gravity.
- Larger K_P gives faster correction.
- Too large can make the estimate noisy or twitchy.

K_I : integral correction

- Helps compensate slow gyro bias.
- Useful when bias is persistent.
- Too large can accumulate wrong correction during motion.

Teaching note

For the first demo, we start with $K_I = 0$ and tune K_P . Add K_I only after we understand gyro bias and drift.

Aspect	Madgwick	Mahony
Correction idea	Gradient descent	Feedback control
Main tuning	β	K_P, K_I
Control analogy	Optimization step	PI controller
Uses quaternion?	Yes	Yes
Uses accelerometer?	Gravity correction	Gravity error feedback
6-axis yaw behavior	Relative / drifts	Relative / drifts

Classroom message

Both filters solve the same practical problem: gyro integration is fast but drifts, so gravity is used to correct roll and pitch. Without a magnetometer, neither filter can correct absolute yaw.

Quaternion to roll-pitch-yaw

After the filter updates $\mathbf{q}$, convert it to human-readable angles.

For $\mathbf{q} = [q_w, q_x, q_y, q_z]^T$:

$$\phi = \text{atan2} (2(q_w q_x + q_y q_z), 1 - 2(q_x^2 + q_y^2))$$

$$\theta = \arcsin (2(q_w q_y - q_z q_x))$$

$$\psi = \text{atan2} (2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2))$$

Names

$$\phi = \text{roll}, \quad \theta = \text{pitch}, \quad \psi = \text{yaw}.$$

Rotation order used by the Python viewer

The Python viewer renders a body-frame triad using the body-to-world rotation matrix

$$\mathbf{R}_{WB} = \mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi).$$

Rotated body axes

$$\mathbf{e}_{x,W} = \mathbf{R}_{WB} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{y,W} = \mathbf{R}_{WB} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{z,W} = \mathbf{R}_{WB} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Why render three arrows?

A single arrow cannot show rotation about its own axis. A three-axis body-frame triad makes roll, pitch, and yaw visible.

What can a 6-axis IMU actually estimate?

Quantity	Corrected by gravity?	Behavior in demo
Roll ϕ	Yes	Stable long-term
Pitch θ	Yes	Stable long-term
Yaw ψ	No	Relative only; drifts

Reason

Gravity gives a vertical reference:

gravity $\Rightarrow$ roll and pitch reference.

But rotation around the vertical direction does not change the gravity vector:

no magnetometer $\Rightarrow$ no absolute yaw reference.

The complete serial demo

ESP32-S3

```
QMI8658 raw registers
-> scale to g and deg/s
-> 100 Hz IMU RTOS task
-> FreeRTOS queue
-> fusion filter
    (Madgwick in our demo;
     Mahony as alternative)
-> roll,pitch,yaw CSV
```

Python viewer

```
read serial line
parse roll,pitch,yaw
R = Rz(yaw) Ry(pitch) Rx(roll)
render Xb, Yb, Zb arrows
press R to reset display zero
```

Example serial output:

```
12.31,-4.62,18.20
12.35,-4.57,18.24
12.40,-4.52,18.28
```

Using the reset key as a coordinate-frame lesson

When we press R, the Python viewer stores the current rotation as a new reference:

$$\mathbf{R}_0 = \mathbf{R}_{current}.$$

After that, the displayed rotation is

$$\mathbf{R}_{display} = \mathbf{R}_0^T \mathbf{R}_{current}.$$

Interpretation

- The IMU did not physically reset.
- The Madgwick filter did not restart.
- We simply chose a new coordinate frame.

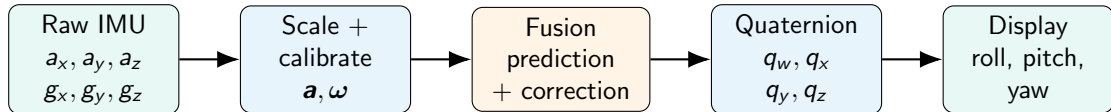
Classroom question

If zero orientation can be redefined by pressing R, what does “zero angle” really mean?

We will turn abstract spatial mathematics into a visible physical system.

- ① Put the board flat and stationary. Observe roll and pitch near zero.
- ② Tilt around the X axis. Which angle changes most?
- ③ Tilt around the Y axis. Which angle changes most?
- ④ Rotate the board flat on the table. Which angle changes most?
- ⑤ Press R. Explain why the displayed zero changed.
- ⑥ Leave the board still for several minutes. Observe yaw drift.
- ⑦ Discuss what extra sensor would be needed for absolute yaw.

Final conceptual summary



Core message

Accelerometer gives a gravity reference. Gyroscope gives fast angular motion. Madgwick and Mahony both fuse these signals into a quaternion. Roll and pitch are corrected; yaw remains relative without a magnetometer.

Appendix: copy-paste Madgwick functions

These appendix functions implement the same 6-axis Madgwick IMU update used in the derivation.

Function contract

$$(q_w, q_x, q_y, q_z)_k, (a_x, a_y, a_z), (\omega_x, \omega_y, \omega_z), \Delta t \longrightarrow (q_w, q_x, q_y, q_z)_{k+1}.$$

- Accelerometer input can be in any consistent unit, such as g ; the function normalizes it.
- Gyroscope input is assumed to be in degrees/s, then converted internally to radians/s.
- The tunable parameter is kept as a global constant: `MADGWICK_BETA`.
- Output quaternion is normalized before return.

Appendix: Copy-Paste Madgwick Functions

6-axis IMU update: accelerometer + gyroscope $\rightarrow$ quaternion

Function contract

At each sample, the function implements the 6-axis Madgwick update

$$(q_w, q_x, q_y, q_z)_k, (a_x, a_y, a_z), (\omega_x, \omega_y, \omega_z), \Delta t \longrightarrow (q_w, q_x, q_y, q_z)_{k+1}.$$

The accelerometer may be in any consistent unit, such as g , because it is normalized internally. The gyroscope input is in degrees/s and is converted internally to radians/s. The tunable parameter is the global constant MADGWICK_BETA.

Python version

```

1  #ifndef MADGWICK_COPY_PASTE_H
2  #define MADGWICK_COPY_PASTE_H
3
4  #include <math.h>
5
6  #define MADGWICK_BETA 0.10f
7  #define MADGWICK_EPS 1.0e-12f
8  #define DEG_TO_RAD_F 0.01745329251994329577f
9  #define RAD_TO_DEG_F 57.295779513082320876f
10
11 typedef struct {
12     float w, x, y, z;
13 } Quaternion;
14
15 typedef struct {
16     float roll, pitch, yaw;
17 } RpyAngles;
18
19 static Quaternion madgwick_update_imu(Quaternion q,
20                                     float ax, float ay, float az,
21                                     float gx, float gy, float gz,
22                                     float dt) {
23     gx *= DEG_TO_RAD_F;
24     gy *= DEG_TO_RAD_F;
25     gz *= DEG_TO_RAD_F;
26
27     /* Gyroscope quaternion derivative. */
28     float qdot_w = 0.5f * (-q.x * gx - q.y * gy - q.z * gz);
29     float qdot_x = 0.5f * ( q.w * gx + q.y * gz - q.z * gy);
30     float qdot_y = 0.5f * ( q.w * gy - q.x * gz + q.z * gx);
31     float qdot_z = 0.5f * ( q.w * gz + q.x * gy - q.y * gx);
32
33     /* Accelerometer correction: use measured gravity direction. */
34     float an2 = ax * ax + ay * ay + az * az;
35     if (isfinite(an2) && an2 > MADGWICK_EPS) {
36         float inv_a = 1.0f / sqrtf(an2);
37         ax *= inv_a;
38         ay *= inv_a;
39         az *= inv_a;
40
41         float f1 = 2.0f * (q.x * q.z - q.w * q.y) - ax;
42         float f2 = 2.0f * (q.w * q.x + q.y * q.z) - ay;
43         float f3 = 1.0f - 2.0f * (q.x * q.x + q.y * q.y) - az;

```

```

44
45     float s_w = -2.0f * q.y * f1 + 2.0f * q.x * f2;
46     float s_x = 2.0f * q.z * f1 + 2.0f * q.w * f2 - 4.0f * q.x * f3;
47     float s_y = -2.0f * q.w * f1 + 2.0f * q.z * f2 - 4.0f * q.y * f3;
48     float s_z = 2.0f * q.x * f1 + 2.0f * q.y * f2;
49
50     float sn2 = s_w * s_w + s_x * s_x + s_y * s_y + s_z * s_z;
51     if (isfinite(sn2) && sn2 > MADGWICK_EPS) {
52         float inv_s = 1.0f / sqrtf(sn2);
53         qdot_w -= MADGWICK_BETA * s_w * inv_s;
54         qdot_x -= MADGWICK_BETA * s_x * inv_s;
55         qdot_y -= MADGWICK_BETA * s_y * inv_s;
56         qdot_z -= MADGWICK_BETA * s_z * inv_s;
57     }
58 }
59
60 q.w += qdot_w * dt;
61 q.x += qdot_x * dt;
62 q.y += qdot_y * dt;
63 q.z += qdot_z * dt;
64
65 float qn2 = q.w * q.w + q.x * q.x + q.y * q.y + q.z * q.z;
66 if ((!isfinite(qn2)) || qn2 <= MADGWICK_EPS) {
67     Quaternion identity = {1.0f, 0.0f, 0.0f, 0.0f};
68     return identity;
69 }
70
71 float inv_q = 1.0f / sqrtf(qn2);
72 q.w *= inv_q;
73 q.x *= inv_q;
74 q.y *= inv_q;
75 q.z *= inv_q;
76 return q;
77 }
78
79 static RpyAngles quaternion_to_rpy(Quaternion q) {
80     RpyAngles rpy;
81
82     float n2 = q.w * q.w + q.x * q.x + q.y * q.y + q.z * q.z;
83     if ((!isfinite(n2)) || n2 <= MADGWICK_EPS) {
84         rpy.roll = 0.0f;
85         rpy.pitch = 0.0f;
86         rpy.yaw = 0.0f;
87         return rpy;
88     }
89
90     float inv_n = 1.0f / sqrtf(n2);
91     q.w *= inv_n;
92     q.x *= inv_n;
93     q.y *= inv_n;
94     q.z *= inv_n;
95
96     float sinr_cosp = 2.0f * (q.w * q.x + q.y * q.z);
97     float cosr_cosp = 1.0f - 2.0f * (q.x * q.x + q.y * q.y);
98     float roll = atan2f(sinr_cosp, cosr_cosp);
99
100     float sinp = 2.0f * (q.w * q.y - q.z * q.x);
101     if (sinp > 1.0f) sinp = 1.0f;
102     if (sinp < -1.0f) sinp = -1.0f;
103     float pitch = asinf(sinp);
104

```

```
105     float siny_cosp = 2.0f * (q.w * q.z + q.x * q.y);
106     float cosy_cosp = 1.0f - 2.0f * (q.y * q.y + q.z * q.z);
107     float yaw = atan2f(siny_cosp, cosy_cosp);
108
109     rpy.roll = roll * RAD_TO_DEG_F;
110     rpy.pitch = pitch * RAD_TO_DEG_F;
111     rpy.yaw = yaw * RAD_TO_DEG_F;
112     return rpy;
113 }
114
115 #endif /* MADGWICK_COPY_PASTE_H */
```

C version

```

1 import math
2
3 MADGWICK_BETA = 0.10
4 MADGWICK_EPS = 1.0e-12
5 DEG_TO_RAD = math.pi / 180.0
6 RAD_TO_DEG = 180.0 / math.pi
7
8
9 def madgwick_update_imu(q, ax, ay, az, gx, gy, gz, dt):
10     """One 6-axis Madgwick IMU update.
11
12     q: (qw, qx, qy, qz), previous unit quaternion.
13     ax, ay, az: accelerometer reading in any consistent unit, e.g. g.
14     gx, gy, gz: gyroscope reading in deg/s.
15     dt: sample period in seconds.
16
17     Returns: (qw, qx, qy, qz), normalized updated quaternion.
18     """
19     qw, qx, qy, qz = q
20
21     gx *= DEG_TO_RAD
22     gy *= DEG_TO_RAD
23     gz *= DEG_TO_RAD
24
25     # Gyroscope quaternion derivative.
26     qdot_w = 0.5 * (-qx * gx - qy * gy - qz * gz)
27     qdot_x = 0.5 * ( qw * gx + qy * gz - qz * gy)
28     qdot_y = 0.5 * ( qw * gy - qx * gz + qz * gx)
29     qdot_z = 0.5 * ( qw * gz + qx * gy - qy * gx)
30
31     # Accelerometer correction: use measured gravity direction.
32     an2 = ax * ax + ay * ay + az * az
33     if math.isfinite(an2) and an2 > MADGWICK_EPS:
34         inv_a = 1.0 / math.sqrt(an2)
35         ax *= inv_a
36         ay *= inv_a
37         az *= inv_a
38
39     f1 = 2.0 * (qx * qz - qw * qy) - ax
40     f2 = 2.0 * (qw * qx + qy * qz) - ay
41     f3 = 1.0 - 2.0 * (qx * qx + qy * qy) - az
42
43     s_w = -2.0 * qy * f1 + 2.0 * qx * f2
44     s_x =  2.0 * qz * f1 + 2.0 * qw * f2 - 4.0 * qx * f3
45     s_y = -2.0 * qw * f1 + 2.0 * qz * f2 - 4.0 * qy * f3
46     s_z =  2.0 * qx * f1 + 2.0 * qy * f2
47
48     sn2 = s_w * s_w + s_x * s_x + s_y * s_y + s_z * s_z
49     if math.isfinite(sn2) and sn2 > MADGWICK_EPS:
50         inv_s = 1.0 / math.sqrt(sn2)
51         qdot_w -= MADGWICK_BETA * s_w * inv_s
52         qdot_x -= MADGWICK_BETA * s_x * inv_s
53         qdot_y -= MADGWICK_BETA * s_y * inv_s
54         qdot_z -= MADGWICK_BETA * s_z * inv_s
55
56     qw += qdot_w * dt
57     qx += qdot_x * dt
58     qy += qdot_y * dt
59     qz += qdot_z * dt

```

```

60
61     qn2 = qw * qw + qx * qx + qy * qy + qz * qz
62     if (not math.isfinite(qn2)) or qn2 <= MADGWICK_EPS:
63         return (1.0, 0.0, 0.0, 0.0)
64
65     inv_q = 1.0 / math.sqrt(qn2)
66     return (qw * inv_q, qx * inv_q, qy * inv_q, qz * inv_q)
67
68
69 def quaternion_to_rpy(q):
70     """Convert quaternion to roll, pitch, yaw in degrees.
71
72     q: (qw, qx, qy, qz), interpreted with the same ZYX convention:
73         R_WB = Rz(yaw) @ Ry(pitch) @ Rx(roll).
74
75     Returns: (roll_deg, pitch_deg, yaw_deg).
76     """
77     qw, qx, qy, qz = q
78
79     n2 = qw * qw + qx * qx + qy * qy + qz * qz
80     if (not math.isfinite(n2)) or n2 <= MADGWICK_EPS:
81         return (0.0, 0.0, 0.0)
82     inv_n = 1.0 / math.sqrt(n2)
83     qw *= inv_n
84     qx *= inv_n
85     qy *= inv_n
86     qz *= inv_n
87
88     sinr_cosp = 2.0 * (qw * qx + qy * qz)
89     cosr_cosp = 1.0 - 2.0 * (qx * qx + qy * qy)
90     roll = math.atan2(sinr_cosp, cosr_cosp)
91
92     sinp = 2.0 * (qw * qy - qz * qx)
93     sinp = max(-1.0, min(1.0, sinp))
94     pitch = math.asin(sinp)
95
96     siny_cosp = 2.0 * (qw * qz + qx * qy)
97     cosy_cosp = 1.0 - 2.0 * (qy * qy + qz * qz)
98     yaw = math.atan2(siny_cosp, cosy_cosp)
99
100    return (roll * RAD_TO_DEG, pitch * RAD_TO_DEG, yaw * RAD_TO_DEG)

```